

KHIZEX | Intelligent Weather Prediction Pipeline — Data Science Intern Assignment

Detailed Build Specification & Deliverables

Program: Khizex Data Science Internship — 1 Week Build Challenge

1. Overview

You are building a proper **time-series forecasting pipeline** for regional weather (temperature, humidity, rainfall) — not a notebook that fits a simple regression and calls it done. The bar here is doing time-series modeling *correctly*: respecting temporal order, engineering real seasonal features, and evaluating with metrics that actually mean something for forecasting, plus explaining *why* the model predicts what it predicts.

2. Technology Stack (Python / Data Science)

- **Language:** Python, with clean, modular, type-hinted code (not just a linear notebook dump).
 - **Core libraries:** Pandas, NumPy for data engineering.
 - **Modeling:** Prophet, ARIMA/SARIMAX (via `statsmodels`), or a properly-configured XGBoost/LightGBM regressor using lagged features — your choice, but it must genuinely account for seasonality/cyclical patterns.
 - **Evaluation/Interpretability:** `scikit-learn` metrics, and **SHAP** (or a built-in feature-importance mechanism) for interpretability.
 - **Optional:** Jupyter notebooks for exploration, but the final pipeline should be organized as reusable modules/scripts, not solely notebook cells.
-

3. Core Requirements

3.1 Time-Series Forecasting Framework

- Implement a genuine time-series/sequence-aware model to forecast temperature, humidity, and/or rainfall:
 - **Prophet** — good for interpretable seasonal decomposition.
 - **ARIMA/SARIMAX** — good if you want to explicitly model autocorrelation and seasonal orders.
 - **XGBoost/LightGBM configured for time series** — acceptable **only** if you properly construct lagged/rolling features (see 3.2) so the model has temporal context; a plain regression on same-day features without lags does not meet this requirement.
- **Simple linear regression with no seasonal/cyclical handling is explicitly disqualifying** — the brief calls this out directly, and it will be checked.

- Justify your model choice in your write-up: why this model for this data's characteristics (seasonality strength, data volume, noise level).

3.2 Advanced Feature Engineering

Build a custom transformation pipeline (your own functions using Pandas/NumPy, not just calling one library's auto-feature tool) that computes:

- **Moving averages** (e.g., 24-hour, 7-day rolling mean).
- **Rolling standard deviations** (volatility signal).
- **Exponential smoothing** features.
- **Time-lagged variables** — explicitly include lags matching the previous 24-hour and 7-day metrics (e.g., `temp_lag_24h`, `temp_lag_7d`).
- **Cyclical seasonal encoding** — day-of-year and hour-of-day must be encoded using **sine and cosine transformations** (e.g., `$\sin(2\pi \times \text{day\_of\_year} / 365)$` , `$\cos(2\pi \times \text{hour} / 24)$`) rather than raw integers, so the model correctly understands that day 365 and day 1 are adjacent, not distant.

3.3 Data Leakage Prevention & Resampling

- Use a **strict time-based train/validation split** — e.g., train on all data before a cutoff date, validate on data after it. **Do not** use a random/shuffled train-test split on this sequential data; explicitly call out in your write-up why a random split would leak future information into training.
- If using cross-validation, use a time-series-appropriate scheme (e.g., expanding-window or rolling-window CV via `TimeSeriesSplit`), not standard k-fold.
- Handle missing sensor readings with a **proper interpolation method** (e.g., time-based linear interpolation, seasonal interpolation, or forward-fill with justification) — explain why your chosen method doesn't distort the historical distribution, and avoid naive mean-imputation across a time series without justification.

3.4 Model Interpretability & Evaluation Metrics

- Evaluate forecasts using metrics appropriate to regression/forecasting, at minimum:
 - **RMSE** (Root Mean Squared Error).
 - **MAPE** (Mean Absolute Percentage Error).
 - Optionally MAE for robustness comparison.
- Report these metrics on the **held-out, time-ordered validation set** — never on training data, and never on a randomly-shuffled split.
- Add an **interpretability layer**:
 - **SHAP values** (for tree-based models) to show which features (lags, rolling stats, seasonal encodings) are driving specific predictions.
 - Or, for Prophet/ARIMA, use the model's native seasonal decomposition/coefficient output to explain which components (trend, seasonality, specific lags) matter most.
- Present at least one visualization showing predicted vs. actual over the validation period, and at least one visualization of feature importance/SHAP summary.

3.5 Code Quality & Structure

- Organize the project into clear modules, for example:
 - `data/` — raw and processed data handling
 - `features/` — feature engineering functions (lags, rolling stats, cyclical encodings)
 - `models/` — model training/prediction wrappers
 - `evaluation/` — metrics and interpretability
 - `notebooks/` — exploration only, not the final deliverable pipeline
 - You may use AI to research specific model configurations or pandas patterns, but the overall pipeline architecture and feature-engineering logic must be understood and structured by you — messy, unstructured, copy-pasted notebooks will be rejected.
-

4. Deliverables

1. Source code repository (modular pipeline as above) with a README covering: data source, feature engineering approach, model choice and why, and how leakage was prevented.
 2. A results report (can be a notebook or a short PDF/markdown) containing:
 - RMSE and MAPE on the time-ordered validation set.
 - Predicted-vs-actual visualization.
 - SHAP/feature-importance or seasonal-decomposition visualization with a short interpretation.
 3. The interpolation/missing-data handling explained with before/after evidence (e.g., a plot showing gaps filled sensibly, not distorted).
-

5. Evaluation Criteria

1. **Correct time-series methodology** — genuine seasonal/cyclical modeling, no disqualifying plain linear regression.
 2. **Feature engineering depth** — real rolling stats, lags, and correct sine/cosine cyclical encoding.
 3. **No data leakage** — strict time-based split, appropriate CV if used.
 4. **Missing-data handling quality** — sensible, justified interpolation.
 5. **Evaluation rigor** — correct metrics (RMSE/MAPE) reported honestly on held-out time-ordered data.
 6. **Interpretability** — a genuine SHAP/feature-importance or decomposition analysis with real interpretation, not just a chart with no explanation.
 7. **Code organization** — modular, reusable pipeline rather than a single sprawling notebook.
-

6. Self-Check Checklist

- ☐ Model is a genuine time-series/seasonal-aware approach (Prophet/ARIMA/SARIMAX/properly-lagged XGBoost).
- ☐ Moving averages and rolling standard deviations implemented.
- ☐ Exponential smoothing features implemented.
- ☐ 24-hour and 7-day lag features implemented.
- ☐ Day-of-year and hour-of-day encoded via sine/cosine transforms.

- ☐ Train/validation split is strictly time-based, not random.
 - ☐ Time-series-appropriate CV used if cross-validation is performed.
 - ☐ Missing sensor data handled via a justified interpolation method.
 - ☐ RMSE and MAPE reported on the held-out time-ordered validation set.
 - ☐ SHAP values or equivalent interpretability analysis included with a written interpretation.
 - ☐ Predicted-vs-actual visualization included.
 - ☐ Code organized into modular components, not a single notebook dump.
-

7. Timeline

- **Day 1:** Data acquisition/cleaning, exploratory analysis, missing-data assessment.
 - **Day 2:** Build feature engineering pipeline (rolling stats, lags, cyclical encodings).
 - **Day 3:** Implement time-based train/validation split and interpolation strategy.
 - **Day 4:** Train the forecasting model(s); initial evaluation.
 - **Day 5:** Tune model, finalize RMSE/MAPE evaluation on validation set.
 - **Day 6:** Build SHAP/interpretability analysis and visualizations.
 - **Day 7:** Write up results report, clean up code into modules, submission.
-